Real-Time Sign Language Recognition

American Sign Language Fingerspelling Recognition from Hand Landmarks

Graduation Project Report

DEPI — Microsoft Machine Learning Program

Team Members

Ahmed Abd El-Ghafar Slama

Ahmed Hassan Mohamed

Yara Ahmed Mohamed Abdelall

Salwa Wael Mohammed Elkordy

2026

Abstract

This project delivers a real-time American Sign Language (ASL) fingerspelling recognition system that reads hand gestures from a webcam and converts them into on-screen text. Rather than classifying raw pixels, the system extracts 21 hand landmarks with Google MediaPipe and feeds a compact, geometry-normalised 63-dimensional feature vector to a Multi-Layer Perceptron (MLP). This landmark-based design makes the model small, fast on CPU, and robust to background, lighting, and skin tone. On a held-out test set of 11,074 samples the model achieves 98.74% accuracy and a 0.986 macro F1-score across the 24 static ASL letters (A–I, K–Y). The trained model is deployed end-to-end as a browser application that runs entirely on the client, and the full pipeline — preprocessing, model, and inference code — is packaged for reuse in a REST/WebSocket backend.

1. Introduction

1.1 Problem Statement

Communication is a daily barrier for deaf and hard-of-hearing people whenever a hearing interpreter is not present. Certified interpreters are scarce and expensive, and everyday interactions — a pharmacy counter, a clinic reception, a bank teller — often have no accessible channel. An automatic, camera-only system that recognises sign language can help bridge this gap instantly and at negligible cost.

1.2 Objective

Build an accurate, real-time system that recognises ASL fingerspelling from a standard webcam and assembles the recognised letters into words, using only a camera — no gloves or sensors.

1.3 Scope

- **In scope:** the 24 static ASL alphabet letters (A–I, K–Y), each a fixed hand pose.
- **Out of scope:** the two motion letters J and Z (they require movement, not a single pose), and the control tokens space / delete / nothing.

2. Dataset

We use the public ASL Alphabet dataset (Kaggle, grassknoted/asl-alphabet): approximately 87,000 colour images of size 200×200, originally spanning 29 classes (A–Z, space, delete, nothing) with about 3,000 images per class. The dataset is well balanced, which avoids the need for class re-weighting.

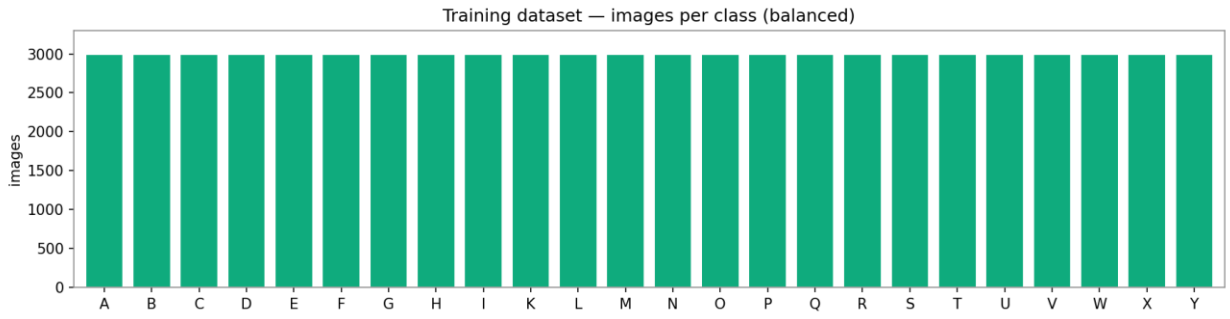

Figure 1. Training images per class — the 24 modelled classes are balanced (~3,000 each).

Because the recognition works on hand landmarks, each image is passed through MediaPipe to detect a hand and extract 21 landmarks; images where no hand is found are skipped. The detection rate varies by pose — closed-fist letters such as N and M are the hardest to detect — but remains high enough across classes to build a large, clean landmark dataset.

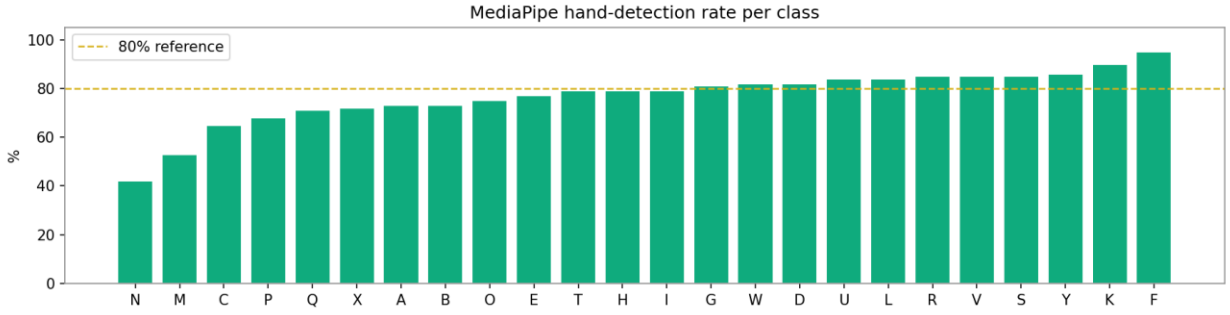

Figure 2. MediaPipe hand-detection rate per class during landmark extraction.

3. Methodology

3.1 Pipeline Overview

Webcam frame → MediaPipe HandLandmarker (21 points) → landmark normalisation (63-D vector) → StandardScaler → MLP classifier → confidence gate → temporal smoothing → letter → word builder.

3.2 Landmark Extraction

MediaPipe Hands returns 21 landmarks per hand as normalised (x, y, z) coordinates. Stacking them produces a 21×3 matrix that is flattened into a 63-dimensional feature vector — the input to every downstream stage.

3.3 Normalisation (why it generalises)

Raw coordinates depend on where the hand is in the frame and how large it appears. We remove both nuisances with a two-step geometric normalisation:

- **Wrist-relative:** subtract the wrist landmark from all points, so the representation no longer depends on the hand's position in the frame (translation invariance).
- **Scale normalisation:** divide all points by the largest wrist-to-landmark distance, so a hand near or far from the camera maps to the same vector (scale invariance).

Because an ASL letter is defined by hand shape — not by screen position or apparent size — removing these factors lets the model learn a much simpler function and generalise across users, cameras, and resolutions. Rotation is deliberately preserved because finger orientation is semantically meaningful in ASL.

3.4 Model Architecture

A small MLP is the right tool for a compact 63-D geometric descriptor: it is accurate, trains in seconds, and runs in well under a millisecond, which keeps the whole system real-time.

Layer	Units	Details
Input	63	flattened 21×3 landmarks
Dense + BatchNorm + ReLU + Dropout	256	dropout 0.35
Dense + BatchNorm + ReLU + Dropout	128	dropout 0.35
Dense + BatchNorm + ReLU	64	—
Dense (Softmax)	24	class probabilities

3.5 Training

- **Preprocessing:** labels encoded with LabelEncoder; features standardised with StandardScaler (fit on the training split only).
- **Split:** stratified 80 / 20 train–test split preserving class proportions.
- **Optimisation:** Adam optimiser, sparse categorical cross-entropy, with EarlyStopping, ReduceLROnPlateau and ModelCheckpoint callbacks.

3.6 Stability at Inference

Two mechanisms turn noisy per-frame predictions into steady output: a confidence gate reports uncertain frames as 'no letter' so wrong letters never flash during hand transitions, and a temporal majority vote over the last few frames removes single-frame flicker. A word builder then commits a letter only after it has been stable for several consecutive frames.

4. Results & Evaluation

The model was evaluated on the held-out test set of 11,074 landmark samples.

Metric	Score
Accuracy	0.9874
Precision (weighted)	0.9876
Recall (weighted)	0.9874
F1-score (weighted)	0.9874
F1-score (macro)	0.9859

The model reaches near-ceiling performance on almost every class. Precision, recall, and F1 are shown per class below.

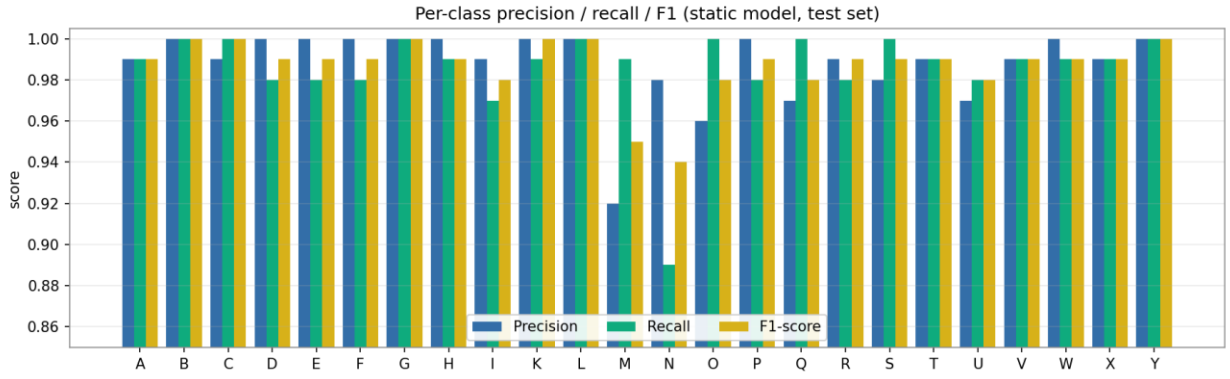

Figure 3. Per-class precision, recall, and F1-score on the test set.

The confusion matrix confirms an almost purely diagonal result. The only notable confusion is between N and M — both are closed-fist poses that differ only by how many fingers are tucked over the thumb, so they are visually similar even to humans. All other classes are separated cleanly.

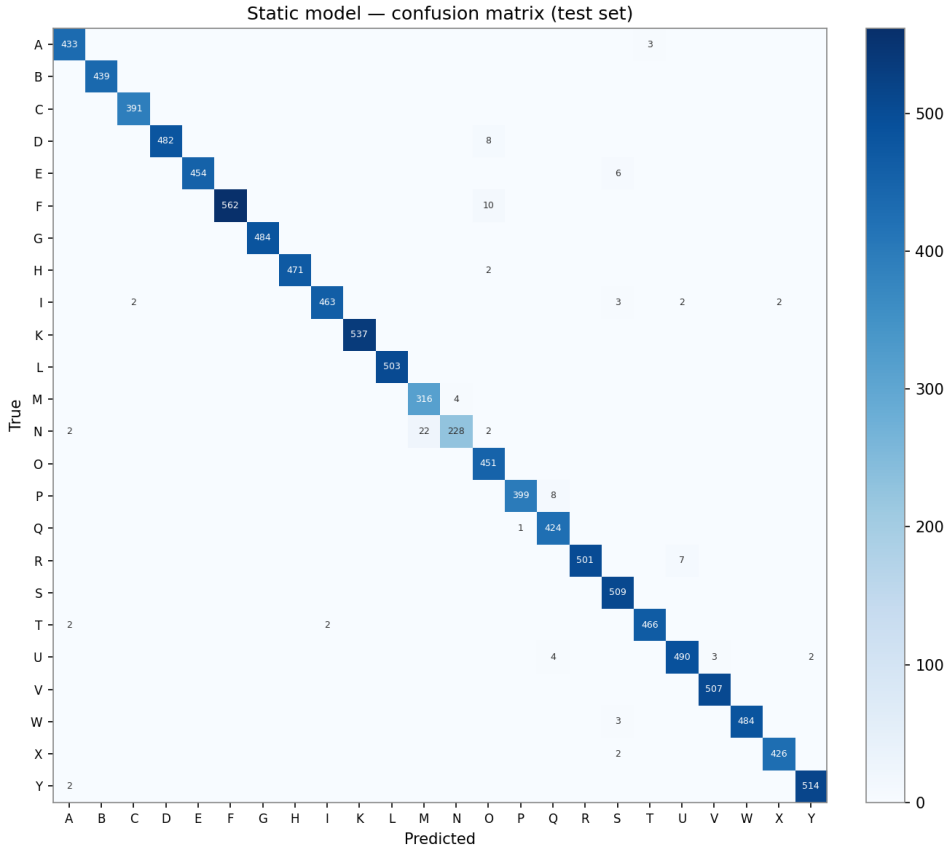

Figure 4. Confusion matrix on the test set (diagonal = correct predictions).

4.1 Full Classification Report

Class	Precision	Recall	F1-score	Support
A	.99	.99	.99	436
B	1.00	1.00	1.00	440
C	.99	1.00	1.00	392
D	1.00	.98	.99	491
E	1.00	.98	.99	461
F	1.00	.98	.99	573
G	1.00	1.00	1.00	486
H	1.00	.99	.99	475
I	.99	.97	.98	476
K	1.00	.99	1.00	540
L	1.00	1.00	1.00	504
M	.92	.99	.95	320
N	.98	.89	.94	255
O	.96	1.00	.98	452
P	1.00	.98	.99	408
Q	.97	1.00	.98	424
R	.99	.98	.99	509
S	.98	1.00	.99	511
T	.99	.99	.99	471
U	.97	.98	.98	502
V	.99	.99	.99	510
W	1.00	.99	.99	491
X	.99	.99	.99	431
Y	1.00	1.00	1.00	516

Weighted average: precision 0.99, recall 0.99, F1 0.99 over 11,074 samples.

5. System Implementation & Deployment

5.1 Reusable ML Modules

All preprocessing and inference logic lives in two pure Python modules — asl_landmarks.py (landmark extraction + normalisation) and asl_inference.py (the translator and word builder). The same code that trained the model is imported verbatim by every deployment target, so there is no drift between the notebook and production.

5.2 Deployment Targets

- **Training notebook:** a single-file Google Colab notebook that downloads the data, extracts landmarks, trains the model, evaluates it, and exports the deployment bundle.
- **Backend API:** a FastAPI service exposing REST and WebSocket endpoints for prediction, plus interactive Swagger documentation.
- **Frontend:** a web client that captures webcam frames, draws the hand skeleton, and shows the letter, confidence, and assembled word.
- **Live in-browser demo:** the model is exported to run entirely in the browser (MediaPipe in WebAssembly and the MLP re-implemented in JavaScript), hosted as a static web app — free, always-on, and privacy-preserving because no video leaves the device.

5.3 Technology Stack

Python, TensorFlow/Keras, MediaPipe (Tasks HandLandmarker), scikit-learn, NumPy, OpenCV, FastAPI, Uvicorn; HTML/CSS/JavaScript, WebSocket, Canvas; Google Colab and Docker.

6. Business Impact & Practical Use

A camera-only, real-time accessibility tool scales instant, low-cost communication where human interpreters cannot. Practical deployments include accessibility kiosks at service counters, interactive ASL-learning apps with instant feedback, healthcare intake support, and captioning for video calls. Because the system runs on landmarks it is cheap to deploy at scale and can run on-device for privacy — a visible, measurable accessibility commitment aligned with national digital-inclusion goals.

7. Limitations & Future Work

- **Scope:** the static alphabet (24 letters); the motion letters J and Z are out of scope.
- **Confusion:** N and M (closed-fist poses) are the main source of the small remaining error.
- **Future work:** add motion-based recognition for J/Z, grow to word- and phrase-level signing, support Arabic Sign Language, and add text-to-speech for two-way conversation.

The system is an assistive aid and is not a replacement for professional interpreters in high-stakes settings such as legal or medical consent.

8. Conclusion

The project demonstrates that a landmark-based approach delivers accurate (98.74%), real-time ASL fingerspelling recognition with a tiny, CPU-friendly model that generalises across users and settings. Combined with confidence gating, temporal smoothing, and a word builder, and deployed as an always-on in-browser application, it is a practical, production-shaped accessibility tool built end-to-end from data to live demo.

References

[1] Kaggle — ASL Alphabet dataset (grassknoted/asl-alphabet).

[2] Google MediaPipe — Hand Landmarker (Tasks) documentation.

[3] TensorFlow / Keras — model training and serialization.

[4] scikit-learn — StandardScaler, LabelEncoder, evaluation metrics.

[5] FastAPI and Hugging Face Spaces — deployment.